Федеральное государственное образовательное бюджетное учреждение высшего образования

«Финансовый университет при Правительстве Российской Федерации»

Факультет «Информационных технологий и анализа больших данных»

Направление «Программная инженерия»

Практическая работа №3

По дисциплине **«Кроссплатформенная разработка»**

На тему **«Реализация приложения Electron»**

Выполнила: студентка 4 курса,

группы ТРПО23-1

очной формы обучения

Бурняшева Елена Александровна.

Преподаватель:

Бузыкова Ю.С.

Москва 2026

Задание 1. Подготовка окружения и инициализация проекта
Работа выполнена на Windows 10. Использовались Visual Studio Code, Git, Node.js и npm.

Версии инструментов:

Инструмент	**Версия**
Node.js в среде разработки	24.16.0
npm	11.13.0
Electron	45.0.0
electron-builder	26.15.3

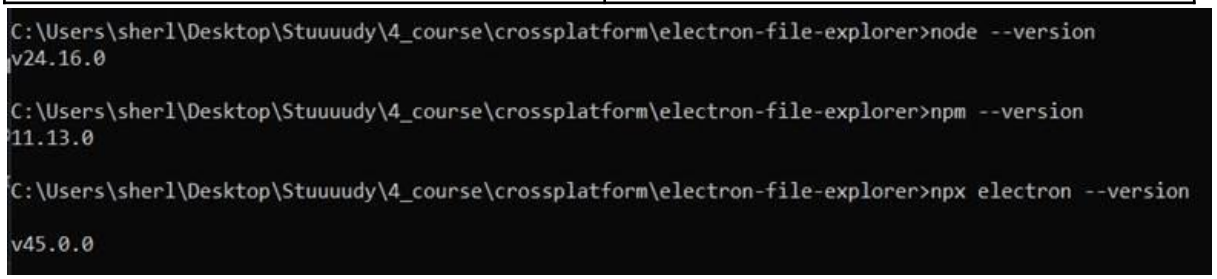


Рисунок 1 — Версии инструментов

Проект создан в каталоге electron-file-explorer.

Для инициализации и установки Electron выполнены команды:

```
npm init -y
 npm install --save-dev electron
```

В package.json указаны название, версия, описание, автор и точка входа main.js. Для запуска определён скрипт start, для сборки — скрипт dist.

```
"scripts": {
   "start": "electron .",
   "dist": "electron-builder --win nsis"
 }
```

Скриншот ниже фиксирует первоначальный манифест; полная итоговая конфигурация приведена в приложении.

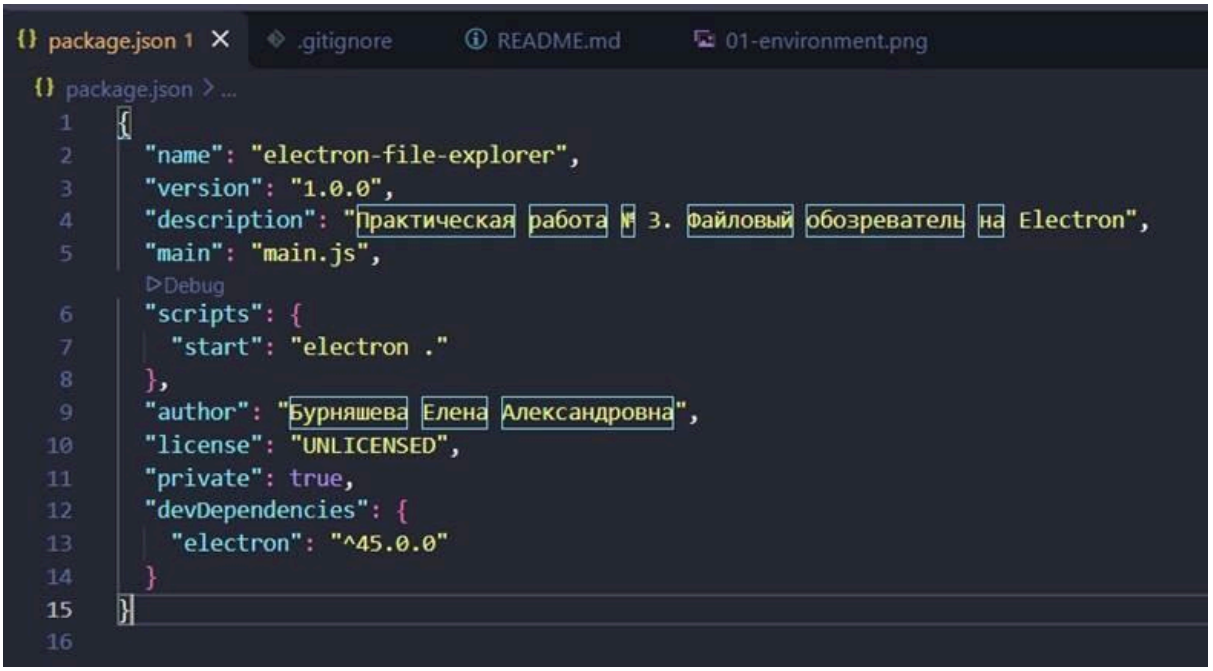


Рисунок 2 — Манифест проекта

Исходный код размещается в репозитории:

https://github.com/burning-helen/crossplatform-practice-03

В .gitignore исключены node_modules/, dist/ и файлы журналов. Файлы package.json и package-lock.json сохраняются в репозитории.



Рисунок 3 — Первый коммит

Задание 2. Минимальное приложение

Структура

Файл	**Назначение**
main.js	Главный процесс: создание окна, меню и обработка системных операций
preload.js	Ограниченный мост между интерфейсом и главным процессом
renderer.js	Обработка действий пользователя и обновление DOM
index.html	Разметка интерфейса
styles.css	Оформление интерфейса

package.json	Метаданные, зависимости и команды проекта

Приложение создаёт окно размером 1000 × 700 пикселей. Минимальный размер окна — 700 × 500 пикселей.

Интерфейс загружается из локального index.html через loadFile. На странице размещены ФИО, группа, номер варианта, кнопка чтения каталога, строка состояния и список результатов.

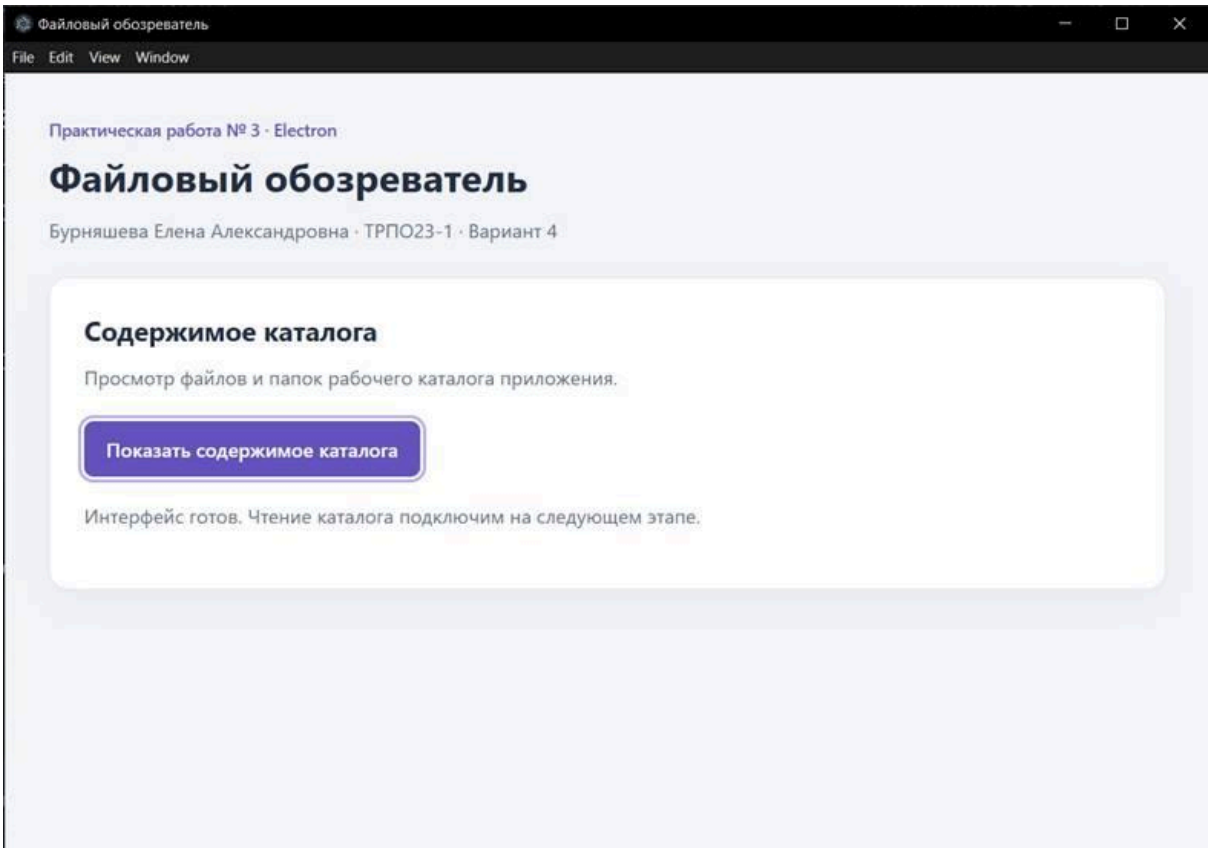


Рисунок 4 — Первое окно

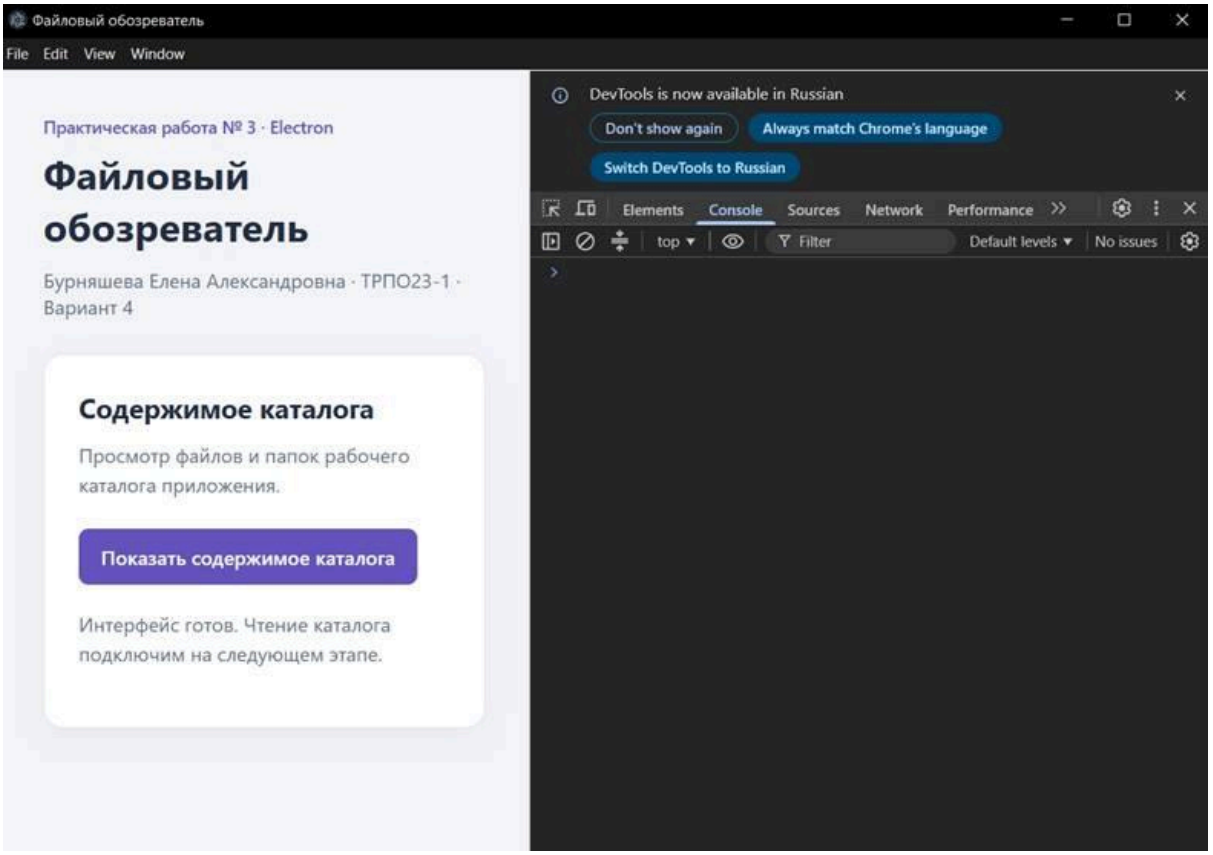


Рисунок 5 — Инструменты разработчика

Жизненный цикл
app.whenReady() возвращает Promise, который разрешается после инициализации Electron. После этого создаются окно и главное меню.

Событие window-all-closed используется для завершения приложения после закрытия всех окон на платформах, отличных от macOS.

На macOS приложение может оставаться запущенным без открытых окон. При событии activate проверяется количество окон и при необходимости создаётся новое.

Задание 3. Preload и IPC: чтение каталога

Разделение ответственности
Файловая операция выполняется в главном процессе. Процесс отрисовки не обращается к модулю fs напрямую.

Последовательность взаимодействия:

1. Пользователь нажимает кнопку.

2. renderer.js вызывает window.api.listDir('.').

3. preload.js отправляет запрос ipcRenderer.invoke('list-dir', dir).

4. main.js обрабатывает его через ipcMain.handle.

5. fs.readdir читает рабочий каталог.

6. Результат возвращается в renderer.js и отображается в списке.

Настройки изоляции
В BrowserWindow явно заданы:

```
webPreferences: {
  preload: path.join(__dirname, 'preload.js'),
  contextIsolation: true,
  nodeIntegration: false,
  sandbox: true
 }
```

nodeIntegration: false отключает прямое предоставление Node.js коду страницы.

contextIsolation: true разделяет JavaScript-контексты preload и веб-страницы.

sandbox: true включает песочницу процесса отрисовки.

Вместо предоставления странице произвольного системного API используется ограниченный мост contextBridge.

Мост preload

```
contextBridge.exposeInMainWorld('api', {
  listDir: (dir) => ipcRenderer.invoke('list-dir', dir),

  onRefreshDirectory: (callback) => {
            if (typeof callback !== 'function') {
            throw new TypeError('Обработчик должен быть функцией.');
            }

            const listener = () => {
            callback();
            };

            ipcRenderer.on('refresh-directory', listener);

            return () => {
    ipcRenderer.removeListener('refresh-directory', listener);
            };
  }
 });
```

Метод listDir выполняет запрос с ответом. Метод onRefreshDirectory подписывает интерфейс на команду меню и возвращает функцию удаления подписки.

Объект IPC-события и весь ipcRenderer в страницу не передаются.

Скриншот ниже относится к этапу чтения каталога до добавления подписки меню; итоговый preload.js приведён в приложении.



Рисунок 6 — Код preload

Чтение каталога в главном процессе
Рабочий каталог фиксируется при запуске:

```
const workingDirectory = process.cwd();
```

Обработчик проверяет, что запрос поступил от главного фрейма локального окна приложения. Допустимым аргументом является только строка ".".

Чтение выполняется асинхронно:

```
const entries = await fs.readdir(workingDirectory, {
  withFileTypes: true
});
```

Результат преобразуется в массив объектов:

```
{
  name: entry.name,
  isDir: entry.isDirectory()
}
```

Сначала выводятся каталоги, затем остальные записи. Внутри групп применяется сортировка по имени.



```
ipcMain.handle('list-dir', async (event, dir) => {
  const senderWindow = BrowserWindow.fromWebContents(event.sender);

  // Принимаем запросы только от главного фрейма нашего локального окна.
  if (
    !senderWindow ||
    event.senderFrame !== event.sender.mainFrame ||
    event.senderFrame.url !== indexUrl
  ) {
    throw new Error('Недопустимый источник запроса.');
  }

  // В этой работе интерфейсу разрешено читать только текущий каталог.
  if (dir !== '.') {
    throw new Error('Разрешено чтение только рабочего каталога.');
  }

  const entries = await fs.readdir(workingDirectory, {
    withFileTypes: true
  });

  return entries
    .map((entry) => ({
      name: entry.name,
      isDir: entry.isDirectory()
    }))
    .sort((a, b) => {
      if (a.isDir !== b.isDir) {
        return a.isDir ? -1 : 1;
      }

      return a.name.localeCompare(b.name, 'ru', {
        numeric: true
      });
    });
});
```

Рисунок 7 — Обработчик IPC

Обновление интерфейса
Функция refreshDirectory() запрашивает данные, очищает предыдущий список и создаёт новые элементы DOM.

Имена записываются через textContent, поэтому не интерпретируются как HTML-разметка.

Для каталогов используется отдельный CSS-класс directory. Во время запроса кнопка блокируется. Ошибки отображаются в строке состояния, после завершения запроса кнопка разблокируется.



```
JS renderer.js
1   const showDirectoryButton = document.getElementById('show-directory');
2   const statusElement = document.getElementById('status');
3   const fileList = document.getElementById('file-list');
4
5   let isLoading = false;
6
7   async function refreshDirectory() {
8     if (isLoading) {
9       return;
10    }
11
12    isLoading = true;
13    showDirectoryButton.disabled = true;
14
15    statusElement.classList.remove('error');
16    statusElement.textContent = 'Чтение каталога...';
17    fileList.replaceChildren();
18
19    try {
20      const entries = await window.api.listDir('.');
21      const fragment = document.createDocumentFragment();
22
23      for (const entry of entries) {
24        const item = document.createElement('li');
25
26        item.className = entry.isDir ? 'directory' : 'file';
27        item.textContent = `${entry.isDir ? '📁' : '📄'} ${entry.name}`;
28
29        fragment.appendChild(item);
30      }
31
32      fileList.appendChild(fragment);
33
34      const directoriesCount = entries.filter((entry) => entry.isDir).length;
35      const filesCount = entries.length - directoriesCount;
36
37      statusElement.textContent = entries.length === 0
38        ? 'Каталог пуст.'
39        : `Всего записей: ${entries.length}. Папок: ${directoriesCount}. Файлов и других записей: ${filesCount}.`;
40    } catch (error) {
41      console.error('Ошибка чтения каталога:', error);
42
43      statusElement.classList.add('error');
44      statusElement.textContent = `Не удалось прочитать каталог: ${error.message}`;
45    } finally {
46      isLoading = false;
47      showDirectoryButton.disabled = false;
48    }
49  }
50
51  showDirectoryButton.addEventListener('click', refreshDirectory);
```

Рисунок 8 — Код renderer

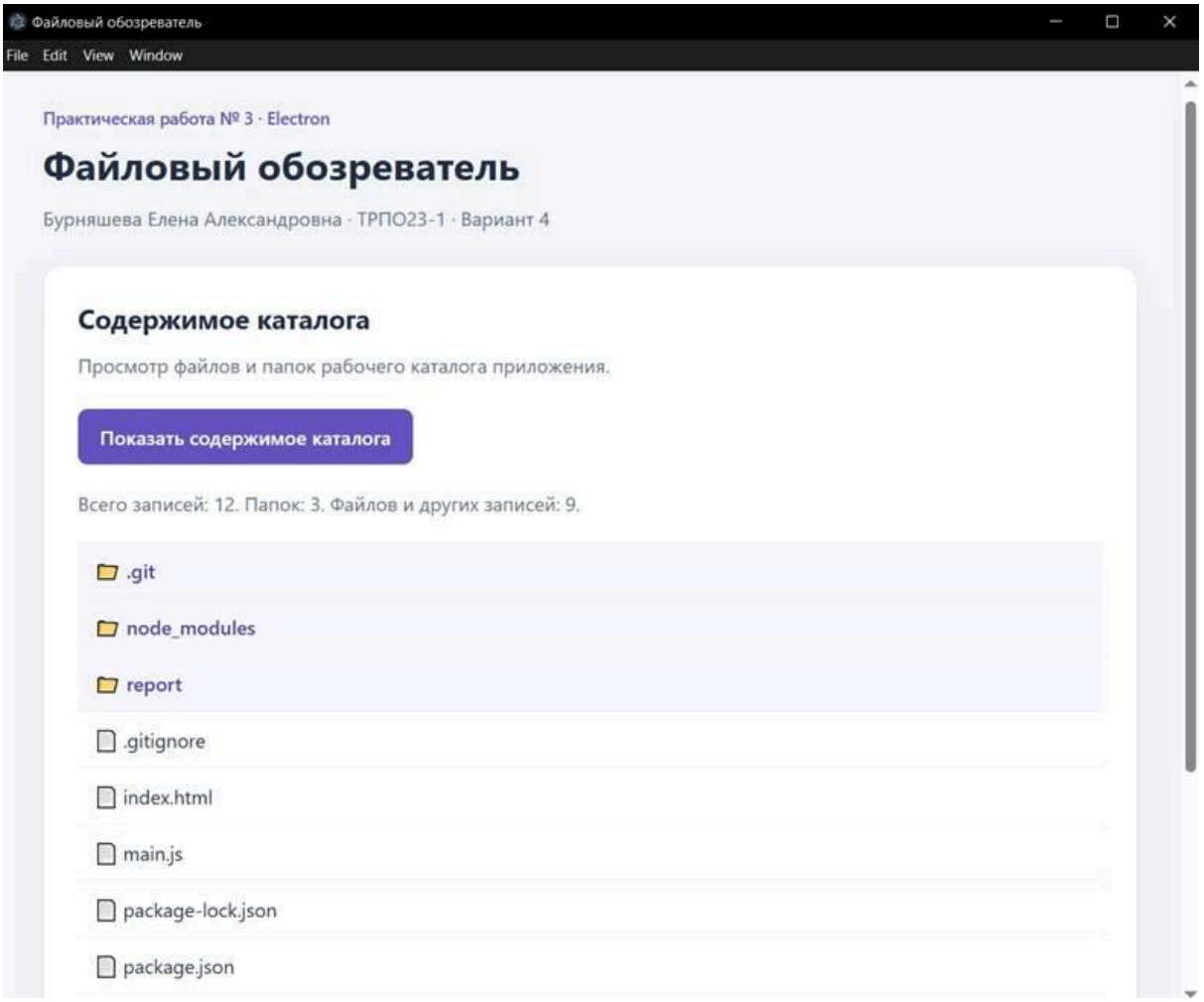


Рисунок 9 — Содержимое каталога

Связь с переносимостью и POSIX
Модуль fs Node.js предоставляет единый интерфейс работы с файловой системой на поддерживаемых платформах.

В приложении используется fs.readdir, поэтому прикладной код не содержит отдельных реализаций перечисления файлов через WinAPI и системные вызовы Unix.

Это иллюстрирует идею переносимого системного API, связанную с POSIX-совместимым программированием. При этом использование Node.js не делает Windows POSIX-совместимой системой и не устраняет все различия файловых систем.

Системные различия скрываются реализацией среды выполнения, но права доступа, правила имён и поведение файловой системы необходимо учитывать при переносе.

Задание 4. Главное меню — вариант 4

Реализованная функция
Добавлено главное меню приложения:

- Файл:
- Обновить список;
- Выйти.
- Вид:
- Инструменты разработчика;
- Обычный масштаб;
- Увеличить;
- Уменьшить.

Меню создаётся через Menu.buildFromTemplate и устанавливается через Menu.setApplicationMenu.

Для macOS дополнительно предусмотрено стандартное меню приложения.

Обработка команды обновления
Пункт «Обновить список» имеет сочетание CmdOrCtrl+R.

Его обработчик выбирает активное окно либо первое доступное окно и отправляет сообщение:

```
targetWindow.webContents.send('refresh-directory');
```

Preload принимает сообщение и вызывает зарегистрированный callback. В renderer.js подписка организована так:

```
const unsubscribeRefresh = window.api.onRefreshDirectory(() => {
  refreshDirectory();
});
```

Таким образом, кнопка и главное меню используют общую функцию refreshDirectory(). Меню инициирует повторное чтение каталога, а не перезагрузку HTML-страницы.

При выгрузке страницы подписка удаляется:

```
window.addEventListener('beforeunload', () => {
  unsubscribeRefresh();
}, { once: true });
```

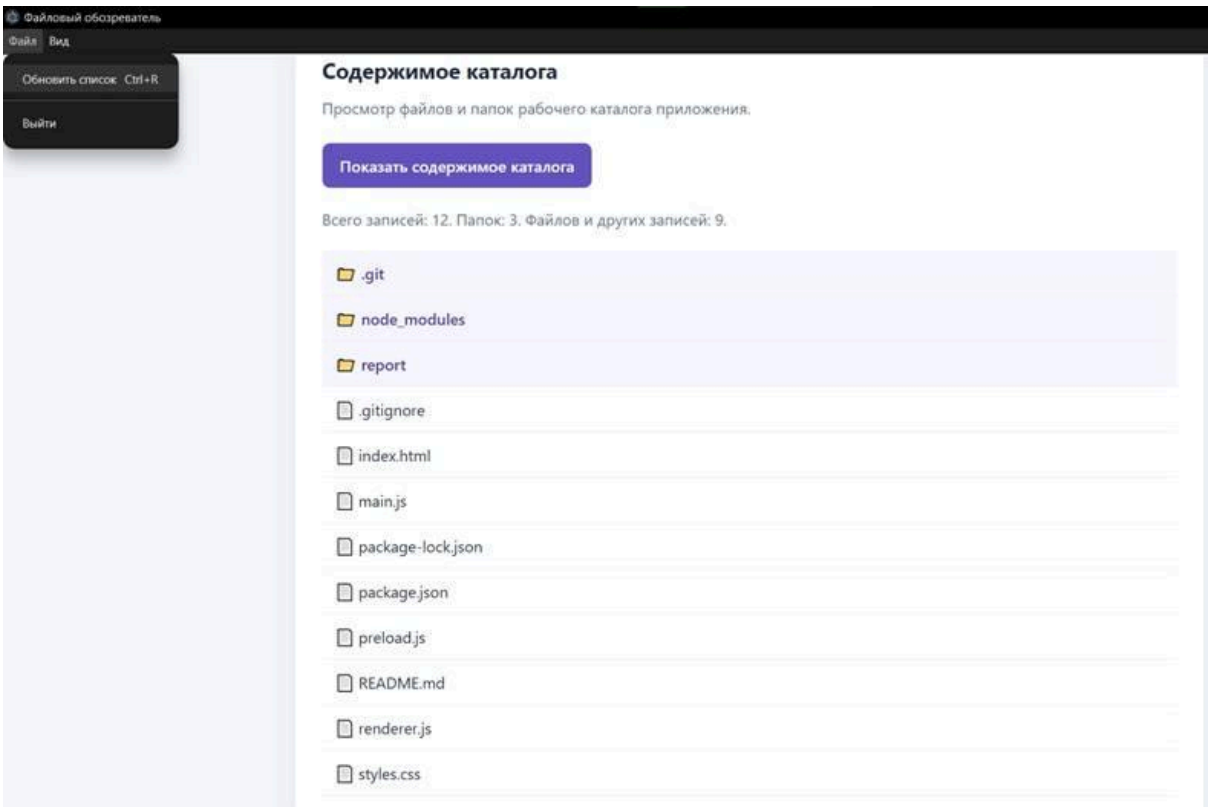


Рисунок 10 — Главное меню

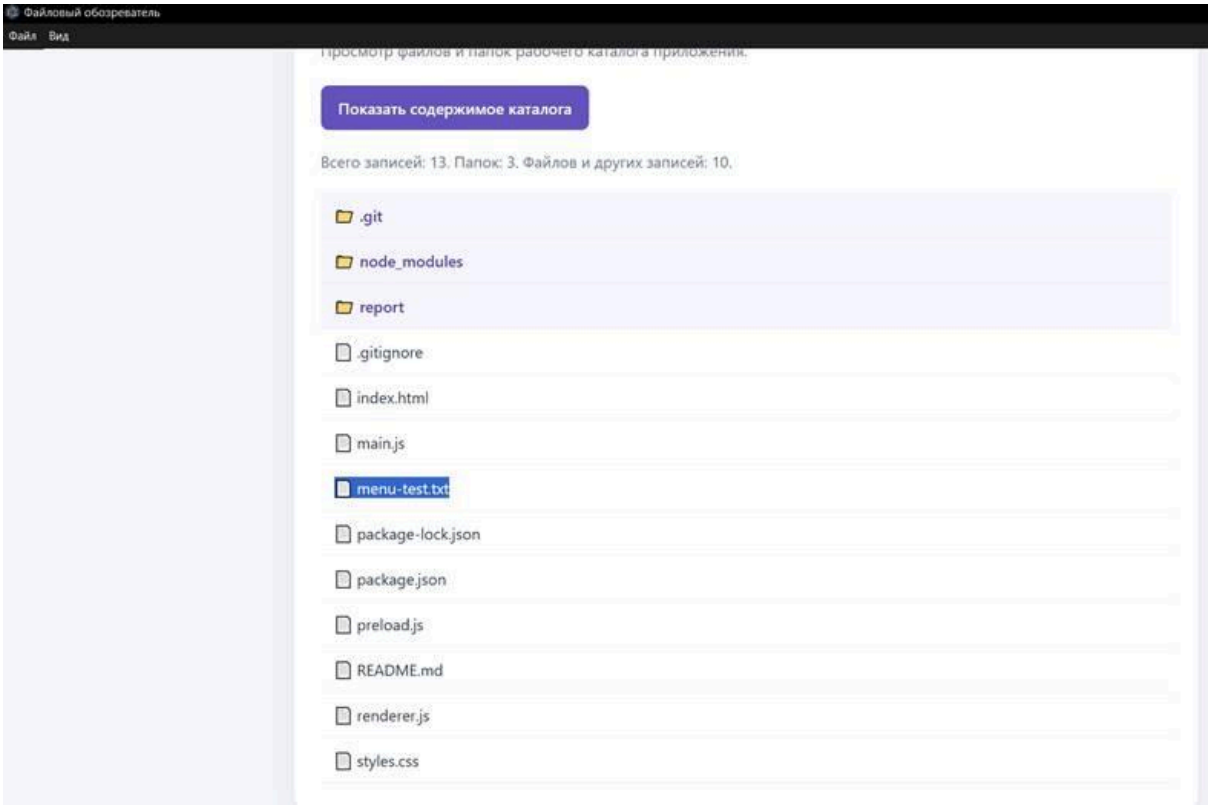


Рисунок 11 — Результат обновления



```
function createApplicationMenu() {
  const template = [
    {
      label: 'Файл',
      submenu: [
        {
          label: 'Обновить список',
          accelerator: 'CmdOrCtrl+R',
          click: (_menuItem, focusedWindow) => {
            const targetWindow =
              focusedWindow || BrowserWindow.getAllWindows()[0];

            if (targetWindow && !targetWindow.isDestroyed()) {
              targetWindow.webContents.send('refresh-directory');
            }
          }
        },
        {
          type: 'separator'
        },
        {
          role: 'quit',
          label: 'Выйти'
        }
      ]
    },
    {
      label: 'Вид',
      submenu: [
        {
          role: 'toggleDevTools',
          label: 'Инструменты разработчика'
        },
        {
          type: 'separator'
        },
        {
          role: 'resetZoom',
          label: 'Обычный масштаб'
        },
        {
          role: 'zoomIn',
          label: 'Увеличить'
        },
        {
          role: 'zoomOut',
          label: 'Уменьшить'
        }
      ]
    }
  ];

  if (process.platform === 'darwin') {
    template.unshift({
      role: 'appMenu'
    });
  }

  const menu = Menu.buildFromTemplate(template);
  Menu.setApplicationMenu(menu);
}
```

Рисунок 12 — Код меню

Источники

- Electron. Process Model: https://www.electronjs.org/docs/latest/tutorial/process-model
- Electron. Inter-Process Communication: https://www.electronjs.org/docs/latest/tutorial/ipc
- Electron. Context Isolation: https://www.electronjs.org/docs/latest/tutorial/context-isolation

- Electron. Menu: https://www.electronjs.org/docs/latest/api/menu
- Node.js. File System: https://nodejs.org/api/fs.html

Дата обращения: 22.10.2026.

Задание 5. Сборка дистрибутива

Инструменты и параметры сборки

Сборка выполнена на Windows 10 для архитектуры x64.

Использованные инструменты:

- Node.js: 24.16.0 — среда разработки.
- npm: 11.13.0.
- Electron: 45.0.0.
- electron-builder: 26.15.3.

Версия Node.js, установленная на компьютере, относится к среде разработки. Версия Node.js, встроенная в Electron, определяется отдельно через process.versions.node.

Для упаковки выбран формат NSIS — установщик Windows.

Команда сборки:

```
npm run dist
```

Результат

В каталоге dist сформирован установщик:

Electron File Explorer Setup 1.0.0.exe

Размер установщика: 112 150 000 байт.

В окне свойств Windows размер отображается как приблизительно 106 МБ. Измерялся размер самого установщика, а не размер установленного приложения и не значение «На диске».

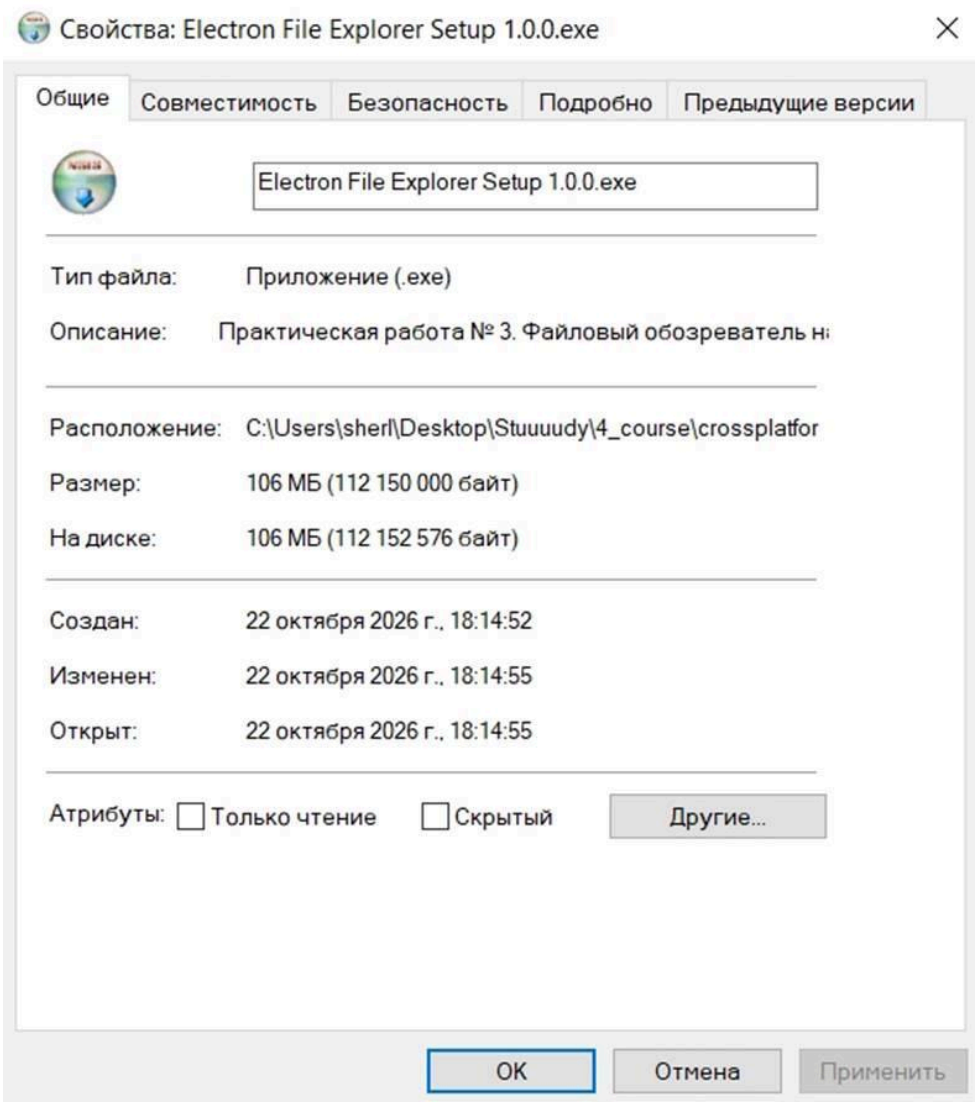


Рисунок 13 — Размер установщика

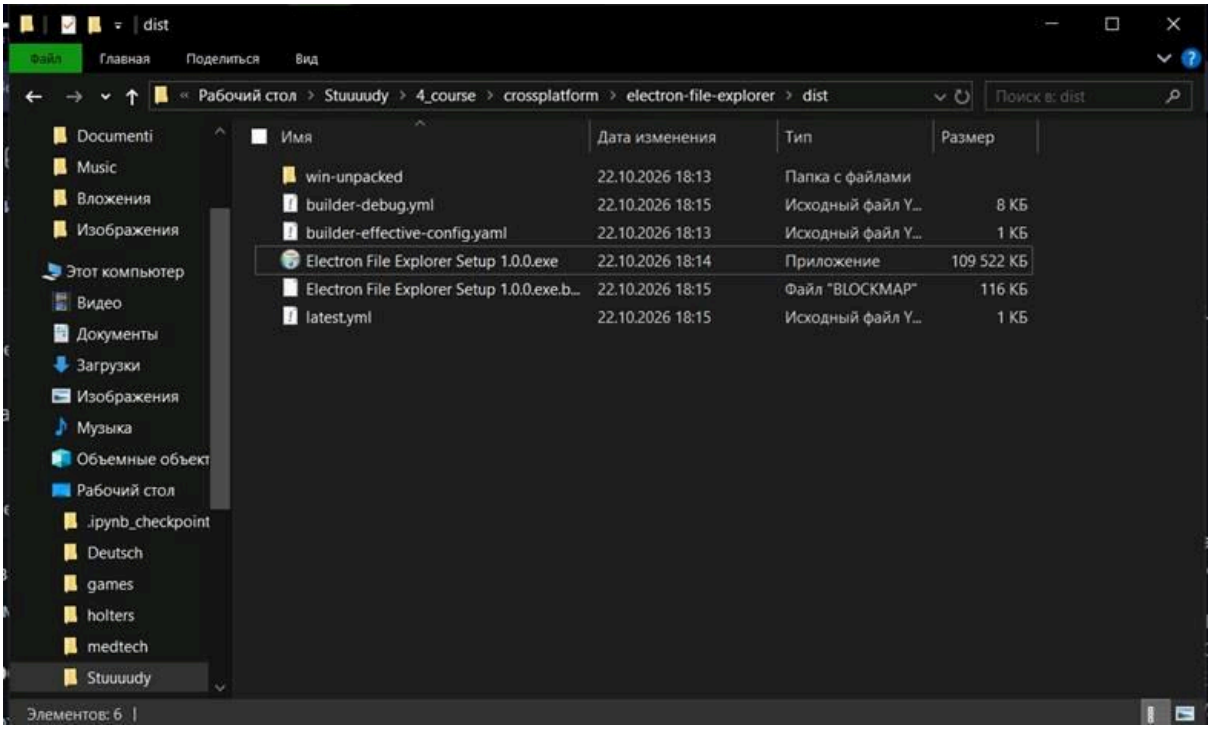


Рисунок 14 — Содержимое каталога dist

Проверка упакованного приложения

Приложение запущено из каталога dist/win-unpacked. Проверены чтение рабочего каталога и обновление списка через пункт главного меню «Обновить список». Обе функции работают корректно.

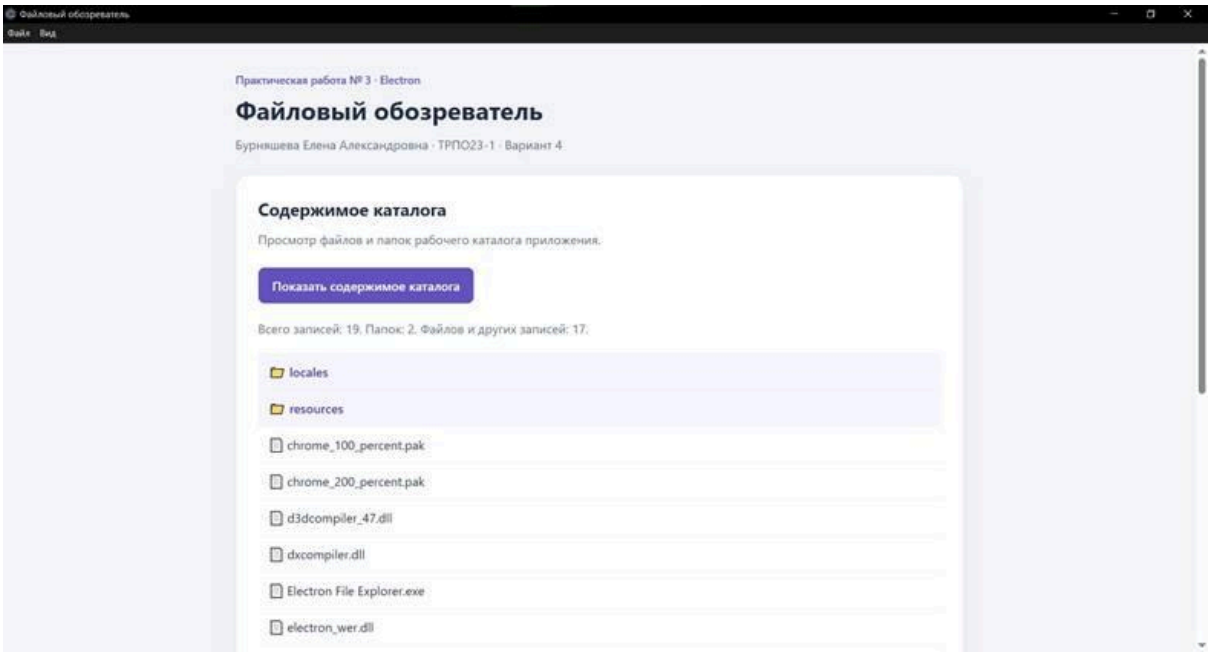


Рисунок 15 — Упакованное приложение

Вывод

Полученный установщик занимает 112 150 000 байт, несмотря на небольшой объём прикладного кода. Это объясняется тем, что приложение распространяется вместе с инфраструктурой Electron, включающей Chromium и Node.js.

По сравнению с небольшим нативным приложением, использующим системные библиотеки интерфейса, такой способ поставки создаёт существенные накладные расходы. Численное сравнение с нативным аналогом в данной работе не выполнялось: отдельный нативный дистрибутив не собирался и не измерялся.

Electron целесообразен для приложений со сложным веб-интерфейсом, когда важны повторное использование веб-технологий и общая кодовая база для нескольких настольных платформ.

Для небольших утилит с жёсткими ограничениями по размеру поставки следует рассмотреть нативную разработку или фреймворки, использующие системный WebView. Окончательный выбор требует проверки доступных API, системных зависимостей и стоимости сопровождения.

Источники

- Electron — официальный репозиторий: https://github.com/electron/electron
- electron-builder — документация: https://www.electron.build/docs/
- electron-builder — NSIS: https://www.electron.build/docs/nsis/

Дата обращения: 22.10.2026.

Ответы на контрольные вопросы

1. Чем Electron отличается от нативной разработки и от Tauri? Каковы его сильные и слабые стороны?

Electron позволяет создавать настольные приложения на HTML, CSS и JavaScript, используя встроенные Chromium и Node.js.

При нативной разработке приложение непосредственно использует инструменты и библиотеки целевой платформы. При этом существуют и переносимые библиотеки для разработки на компилируемых языках, поэтому нативный исполняемый файл не обязательно означает отдельную кодовую базу для каждой ОС.

Tauri также позволяет использовать веб-интерфейс, но отображает его в системном WebView, а нативная часть обычно реализуется на Rust.

Сильные стороны Electron:

- использование веб-технологий;
- общая прикладная кодовая база для настольных платформ;
- развитая экосистема;
- поставка определённой версии Chromium вместе с приложением.

Ограничения:

- большой базовый размер дистрибутива;
- накладные расходы браузерной инфраструктуры;
- необходимость регулярного обновления Electron и зависимостей;
- необходимость безопасно организовать доступ интерфейса к системным возможностям.

2. Что такое main process и renderer process? Сколько экземпляров каждого типа существует?

Главный процесс, main process, управляет жизненным циклом приложения, создаёт окна, организует меню и выполняет привилегированные операции.

В одном экземпляре приложения Electron существует один главный процесс. В нашем проекте его точкой входа является main.js.

Процессы отрисовки, renderer processes, отображают веб-интерфейс. В базовой модели каждому окну BrowserWindow соответствует отдельный процесс отрисовки. В сложном приложении Chromium может создавать дополнительные процессы.

Поэтому Electron-приложение не обязательно состоит ровно из двух процессов: кроме главного и процессов отрисовки могут существовать процессы GPU, служебные процессы и другие.

3. Какую роль играет package.json? Какие поля нужны для упаковки?

package.json описывает проект: его название, версию, зависимости, команды и точку входа.

В работе используются:

- name — имя пакета;
- version — версия приложения;
- description — описание;
- author — автор;
- main — файл главного процесса;
- scripts — команды запуска и сборки;
- devDependencies — зависимости разработки;
- build — настройки electron-builder.

Для подготовки дистрибутива должны быть корректно заполнены метаданные name и version, задана существующая точка входа main. Поля description и author требуются в рамках методички и используются средствами упаковки.

В проекте main указывает на main.js, npm start запускает Electron, а npm run dist формирует установщик Windows.

4. Почему в renderer отключён прямой доступ к Node.js? Что означают contextIsolation и nodeIntegration?

Интерфейс обрабатывает веб-содержимое. Если предоставить ему полный доступ к Node.js, ошибка обработки недоверенных данных или внедрение скрипта может привести к выполнению системных операций с правами приложения.

nodeIntegration: false отключает предоставление Node.js обычному коду страницы.

contextIsolation: true разделяет JavaScript-контексты страницы и preload. Страница не получает прямого доступа к внутренним объектам preload.

Эти настройки решают разные задачи и применяются совместно. В работе дополнительно включена песочница sandbox: true.

Разрешённые операции предоставляются через ограниченный API contextBridge. Наличие изоляции не отменяет проверку аргументов и источников IPC-запросов.

5. Что такое preload и какова роль contextBridge.exposeInMainWorld?

Preload — скрипт, который выполняется перед загрузкой кода страницы в процессе отрисовки. Он имеет доступ к разрешённым Electron API, а при включённой песочнице — к ограниченному набору возможностей среды Node.js.

contextBridge.exposeInMainWorld публикует выбранный API из изолированного контекста preload в контекст страницы.

В работе опубликован объект window.api с методами:

- listDir — запрос содержимого рабочего каталога;
- onRefreshDirectory — подписка на команду обновления из меню.

Странице не передаются весь ipcRenderer и произвольные файловые операции.

6. Как работают invoke/handle? Чем они отличаются от send/on?

ipcRenderer.invoke отправляет запрос из процесса отрисовки и возвращает Promise.

ipcMain.handle регистрирует обработчик в главном процессе. Возвращённое обработчиком значение становится результатом Promise. Ошибка обработчика приводит к отклонению Promise.

В нашем приложении invoke/handle используются для чтения каталога: интерфейс запрашивает данные и ожидает массив записей.

Пара ipcRenderer.send / ipcMain.on предназначена для событийного обмена и не возвращает результат обработчика как Promise. При необходимости ответ организуется отдельно.

Для передачи команды меню из main в renderer используется webContents.send, а в preload сообщение принимается через ipcRenderer.on.

7. Чем различаются win.loadFile и win.loadURL? Что используется в работе?

win.loadFile загружает локальный HTML-файл.

win.loadURL загружает содержимое по URL, например страницу сервера разработки или удалённого сайта.

В работе используется loadFile с абсолютным путём к index.html, сформированным через path.join(__dirname, 'index.html').

Интерфейс входит в состав приложения, поэтому для его загрузки не требуется отдельный веб-сервер. Обращения к файловой системе выполняются через IPC, а не напрямую из HTML-страницы.

8. Как обрабатывается жизненный цикл и почему macOS — особый случай?

app.whenReady() — метод, возвращающий Promise готовности Electron, а не отдельное событие с таким именем. После его разрешения создаются окно и меню.

Событие window-all-closed возникает после закрытия всех окон. В проекте оно вызывает app.quit(), если платформа не является macOS.

На macOS закрытие последнего окна обычно не означает завершение приложения. Поэтому приложение остаётся запущенным.

При событии activate проверяется количество открытых окон. Если их нет, вызывается createWindow().

9. Как electron-builder создаёт дистрибутив? Почему не все сборки возможны на любой ОС?

electron-builder читает метаданные проекта и настройки build, подготавливает Electron для выбранной платформы и архитектуры, включает прикладные файлы и создаёт выбранный формат поставки.

В работе используется цель NSIS для Windows x64. Результаты сохраняются в каталог dist.

Возможности межплатформенной сборки зависят от формата пакета, системных инструментов, нативных зависимостей и требований подписи.

Например, подпись приложения macOS выполняется на macOS. Поэтому наличие общей кодовой базы не означает возможность подготовить любой подписанный дистрибутив на любой системе.

10. Почему дистрибутив Electron большой? Как можно уменьшить размер?

В дистрибутив включаются Chromium, Node.js и другие компоненты среды Electron, а не только JavaScript, HTML и CSS приложения.

В нашей работе размер установщика составил 112 150 000 байт, что Windows отображает приблизительно как 106 МБ.

Для уменьшения размера можно:

- исключить ненужные файлы через настройки files;
- не включать отчёт, скриншоты и тестовые данные;
- оптимизировать изображения и другие ресурсы;
- удалять неиспользуемые зависимости;
- использовать подходящие параметры сжатия установщика;
- при необходимости проверить возможность исключения неиспользуемых локализаций.

ASAR объединяет файлы приложения в архив, но сам по себе не является сжатием и не устраняет размер Chromium и Node.js.

Если минимальный размер является определяющим требованием, следует рассмотреть другую архитектуру приложения.

11. Как чтение каталога связано с POSIX-совместимым программированием?

Приложение использует единый API fs.readdir, предоставляемый Node.js, вместо отдельных реализаций перечисления файлов для Windows и Unix-подобных систем.

Это иллюстрирует идею переносимого системного интерфейса: прикладной код обращается к общей операции, а среда выполнения организует её реализацию для целевой платформы.

Однако fs.readdir — API Node.js, а не прямое доказательство POSIX-совместимости операционной системы. Windows от использования этого метода не становится POSIX-совместимой.

Различия путей, прав доступа, регистра символов и файловых систем могут сохраняться и требуют проверки.

12. Что возвращает process.versions и как это помогает диагностике?

process.versions — объект со строковыми версиями Node.js и компонентов среды выполнения.

В Electron он содержит в том числе:

- process.versions.electron — версию Electron;
- process.versions.chrome — версию Chromium;

- process.versions.node — версию встроенного Node.js.

Эти данные помогают проверить доступность API, совместимость нативных модулей и соответствие сообщения об ошибке конкретной версии среды выполнения.

Версия из команды node --version относится к Node.js, установленному в среде разработки. Она не обязана совпадать с process.versions.node внутри запущенного Electron.

В приложении с изолированным интерфейсом нужные значения следует передавать через ограниченный API preload или IPC, а не включать полный доступ страницы к Node.js.

Источники

1. Методические указания к практической работе № 3.

2. Electron. Process Model. https://www.electronjs.org/docs/latest/tutorial/process-model

3. Electron. Using Preload Scripts. https://www.electronjs.org/docs/latest/tutorial/tutorial-preload

4. Electron. Context Isolation. https://www.electronjs.org/docs/latest/tutorial/context-isolation

5. Electron. Inter-Process Communication. https://www.electronjs.org/docs/latest/tutorial/ipc

6. Electron. BrowserWindow. https://www.electronjs.org/docs/latest/api/browser-window

7. Electron. app. https://www.electronjs.org/docs/latest/api/app

8. Electron. process. https://www.electronjs.org/docs/latest/api/process

9. Electron. ASAR Archives. https://www.electronjs.org/docs/latest/tutorial/asar-archives

10. electron-builder. Documentation. https://www.electron.build/docs/

11. electron-builder. Multi Platform Build.
https://www.electron.build/docs/features/multi-platform-build/

12. Node.js. File System. https://nodejs.org/api/fs.html

Дата обращения: 22.10.2026.

Общий вывод

В ходе практической работы разработано настольное приложение «Файловый обозреватель» на Electron. Создано окно BrowserWindow, подготовлен интерфейс и организовано разделение ответственности между главным процессом, preload-скриптом и процессом отрисовки.

Чтение рабочего каталога выполняется в главном процессе средствами fs.readdir. Интерфейс получает результаты через ограниченный API contextBridge и механизм ipcRenderer.invoke / ipcMain.handle. Для окна включены contextIsolation и sandbox, а прямой доступ страницы к Node.js отключён.

По варианту 4 реализовано главное меню с пунктом «Обновить список». Кнопка интерфейса и пункт меню используют одну функцию обновления, что исключает дублирование логики отображения.

Средствами electron-builder сформирован установщик NSIS для Windows x64. Его размер составил 112 150 000 байт. Результат показывает, что небольшой объём прикладного кода не

означает небольшой дистрибутив: вместе с приложением поставляются компоненты Electron, включая Chromium и Node.js.

Получен практический опыт создания Electron-приложения, организации IPC, использования системного меню, упаковки и сопровождения проекта в Git.

Приложение. Итоговые исходные файлы

Листинги соответствуют переданным файлам проекта. Переносы длинных строк при отображении в документе не изменяют исходный код.

main.js

```
const { app, BrowserWindow, ipcMain, Menu } = require('electron');
 const path = require('node:path');
 const fs = require('node:fs/promises');
 const { pathToFileURL } = require('node:url');

 const indexPath = path.join(__dirname, 'index.html');
 const indexUrl = pathToFileURL(indexPath).href;

 // Читаем только рабочий каталог, из которого запущено приложение.
 const workingDirectory = process.cwd();

 function createApplicationMenu() {
   const template = [
                {
                label: 'Файл',
                submenu: [
                {
          label: 'Обновить список',
                accelerator: 'CmdOrCtrl+R',
                click: (_menuItem, focusedWindow) => {
                const targetWindow =
                focusedWindow || BrowserWindow.getAllWindows()[0];

                if (targetWindow && !targetWindow.isDestroyed()) {
              targetWindow.webContents.send('refresh-directory');
                }
                },
                {
                type: 'separator'
                },
                {
                role: 'quit',
                label: 'Выйти'
                }
                ]
                },
                {
                label: 'Вид',
                submenu: [
                {
                role: 'toggleDevTools',
                label: 'Инструменты разработчика'
                },
                {
                type: 'separator'
                },
                {
                role: 'resetZoom',
                label: 'Обычный масштаб'
```

```
            },
            {
            role: 'zoomIn',
            label: 'Увеличить'
            },
            {
            role: 'zoomOut',
            label: 'Уменьшить'
            }
            ]
            }
  ];

  if (process.platform === 'darwin') {
            template.unshift({
            role: 'appMenu'
            });
  }

  const menu = Menu.buildFromTemplate(template);
  Menu.setApplicationMenu(menu);
}

function createWindow() {
  const win = new BrowserWindow({
            width: 1000,
            height: 700,
            minWidth: 700,
            minHeight: 500,
            title: 'Файловый обозреватель — Практическая работа № 3',
            webPreferences: {
            preload: path.join(__dirname, 'preload.js'),
            contextIsolation: true,
            nodeIntegration: false,
            sandbox: true
            }
  });

  win.loadFile(indexPath);

  // Не разрешаем странице открывать новые окна.
  win.webContents.setWindowOpenHandler(() => ({
            action: 'deny'
  }));

  // Не разрешаем переходить с локального интерфейса на другие страницы.
  win.webContents.on('will-navigate', (event) => {
            event.preventDefault();
  });
}

ipcMain.handle('list-dir', async (event, dir) => {
  const senderWindow = BrowserWindow.fromWebContents(event.sender);

  // Принимаем запросы только от главного фрейма нашего локального окна.
  if (
            !senderWindow ||
    event.senderFrame !== event.sender.mainFrame ||
            event.senderFrame.url !== indexUrl
  ) {
            throw new Error('Недопустимый источник запроса.');
  }

  // В этой работе интерфейсу разрешено читать только текущий каталог.
  if (dir !== '.') {
            throw new Error('Разрешено чтение только рабочего каталога.');
```

```
  }

  const entries = await fs.readdir(workingDirectory, {
        withFileTypes: true
  });

  return entries
        .map((entry) => ({
        name: entry.name,
        isDir: entry.isDirectory()
        }))
    .sort((a, b) => {
        if (a.isDir !== b.isDir) {
        return a.isDir ? -1 : 1;
        }

        return a.name.localeCompare(b.name, 'ru', {
        numeric: true
        });
        });
});

app.whenReady().then(() => {
  createWindow();
  createApplicationMenu();

  app.on('activate', () => {
        if (BrowserWindow.getAllWindows().length === 0) {
        createWindow();
        }
  });
});

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') {
        app.quit();
  }
});
```

preload.js

```
const { contextBridge, ipcRenderer } = require('electron');

contextBridge.exposeInMainWorld('api', {
  listDir: (dir) => ipcRenderer.invoke('list-dir', dir),

  onRefreshDirectory: (callback) => {
        if (typeof callback !== 'function') {
        throw new TypeError('Обработчик должен быть функцией.');
        }

        const listener = () => {
        callback();
        };

        ipcRenderer.on('refresh-directory', listener);

        return () => {
    ipcRenderer.removeListener('refresh-directory', listener);
        };
  }
});
```

renderer.js

```
const showDirectoryButton = document.getElementById('show-directory');
 const statusElement = document.getElementById('status');
 const fileList = document.getElementById('file-list');

 let isLoading = false;

 async function refreshDirectory() {
  if (isLoading) {
            return;
  }

  isLoading = true;
  showDirectoryButton.disabled = true;

  statusElement.classList.remove('error');
  statusElement.textContent = 'Чтение каталога...';
  fileList.replaceChildren();

  try {
            const entries = await window.api.listDir('.');
            const fragment = document.createDocumentFragment();

            for (const entry of entries) {
            const item = document.createElement('li');

            item.className = entry.isDir ? 'directory' : 'file';
            item.textContent = `${entry.isDir ? '📁' : '📄'} ${entry.name}`;

            fragment.appendChild(item);
            }

            fileList.appendChild(fragment);

            const directoriesCount = entries.filter((entry) => entry.isDir).length;
            const filesCount = entries.length - directoriesCount;

            statusElement.textContent = entries.length === 0
            ? 'Каталог пуст.'
            : `Всего записей: ${entries.length}. Папок: ${directoriesCount}. Файлов и других записей: ${filesCount}.`;
  } catch (error) {
            console.error('Ошибка чтения каталога:', error);

            statusElement.classList.add('error');
            statusElement.textContent = `Не удалось прочитать каталог: ${error.message}`;
  } finally {
            isLoading = false;
            showDirectoryButton.disabled = false;
  }
 }

 showDirectoryButton.addEventListener('click', refreshDirectory);

 const unsubscribeRefresh = window.api.onRefreshDirectory(() => {
  refreshDirectory();
 });

 window.addEventListener('beforeunload', () => {
  unsubscribeRefresh();
 }, { once: true });
```

package.json

```
{
  "name": "electron-file-explorer",
  "version": "1.0.0",
  "description": "Практическая работа № 3. Файловый обозреватель на Electron",
  "main": "main.js",
  "scripts": {
		"start": "electron .",
		"dist": "electron-builder --win nsis"
  },
  "author": "Бурняшева Елена Александровна",
  "license": "UNLICENSED",
  "private": true,
  "build": {
		"appId": "ru.fa.burnyasheva.fileexplorer",
		"productName": "Electron File Explorer",
		"directories": {
		"output": "dist"
		},
		"files": [
    "main.js",
		"preload.js",
		"renderer.js",
		"index.html",
		"styles.css",
		"package.json"
		],
		"win": {
		"target": "nsis"
		},
		"nsis": {
		"oneClick": false,
		"perMachine": false,
    "allowToChangeInstallationDirectory": true
		}
  },
  "devDependencies": {
		"electron": "^45.0.0",
		"electron-builder": "^26.15.3"
  }
}
```